

DIGITAL WAREHOUSE RECEIPTS

First name: Pavan Kumar
Last name: Chella
email: pchella@buffalo.edu
person number: 50468328

First name: Sai Rohit
Last name: Singampalli
email: ssingamp@buffalo.edu
person number: 50496359

Issues Addressed

- **Enable Global trade by providing access to open markets:**
In the case of goods, it could be a challenge to sell beyond a certain distance and reach out to more distant players who might be offering a better price for the product. Usually, sellers are limited in reaching out to buyers who are far away. We help them to reach out to maximum number players making it open and accessible to everyone who would like to participate in such trades.
- **LOT size issues:**
In commodity trades there is fixed minimum quantity to trade below which a player cannot participate. Each token has no limitations for quantity they represent trade. So, this gives small players an opportunity to trade in the market.
- **Price manipulation and Lower transactional charges:**
There are high transaction charges when goods are traded on commodity exchanges. The traders end up paying a lot of transaction fees and exchange fees which could be easily replaced by the gas, which is minimal compared to the charges levied by the exchange platforms. Unlike commodity markets, these are less subjected to price manipulation as each token is independent of each other.
- **Transportation of goods:**
To display the product to the buyers the farmers might have to carry the goods so that they could create trust on the quality of the product. Since here the tokens are created only by warehouse authorities they are stored in the warehouse throughout the trade and decrease the spoilage of the goods.
- **Safe and Guaranteed Delivery:**
As the token is created by the warehouse it guarantees the possession of the product. This builds confidence in buyers and ensures trust regarding the product and move forward to trade. Warehouses ensure proper storage of the goods in necessary condition.

Abstract

Warehouse receipts are guaranteed documents that play a vital role in ensuring the delivery of goods at warehouses. These receipts hold a financial value depending on the material they represent on paper. They are used as collateral, for trading and getting delivery. The paper form of these receipts limits their exposure and trade on the open market. Our solution tokenizes these receipts and launches them on the web3 platform for trading. We aim to bring transparency in the system about trades happening on the commodity market and decrease the transactional charges, unlike the commodity markets. These tokens can be created only by authorized warehouses for the seller. A seller gets more options to sell from anywhere. It reduces the unnecessary transportation of goods and allows more players with their products. Big business players also can participate in those trades to explore the distant options available to them globally.

Digital Asset and Token

The digital asset used in this project is a Digital copy of the warehouse receipt generated by the warehouse regarding the possession of physical goods. The token ID is the token which is assigned to each warehouse receipt generated on the web 3 platform. It is autogenerated and unique for each warehouse receipt on the platform making them a non-fungible token. It is secure, cannot be lost, or stolen and only be destroyed by user. It provides better transparency and makes traders more accountable since all the transactions are recorded on the immutable ledger on the blockchain. This can be used for auditing purposes in the future in case of any sort of fraud. This also has the same benefits as the traditional way of trading goods and commodities, making it much easier and accessible. We have used ERC 721 standards for our smart contract and created receipts which are non-fungible tokens as each receipt has a unique token number and cannot be exchanged with other receipts. They could trade on the web 3 platform and after the delivery it could be used to get delivery from the warehouse by the owner who holds the receipt at the time of delivery.

Quad Chart

Use case: Warehouse Receipt Problem Statement: A blockchain based trade system for goods stored in Warehouses based on the warehouse receipts.	Issues with existing system 1. High transaction fees on exchanges 2. Less accessibility for remote traders 3. Price manipulation by big traders 4. Transportation of Goods for trading 5. No records for the goods stored in warehouses for auditing
Proposed blockchain-based solution 1. Smart contract acts as medium to trade warehouse receipts on the web3 platform 2. A moderator launches the smart contract to facilitate the trades and he registers a warehouse role to user. 3. A seller who has a product first gets his product stored in the warehouse, and then warehouse provides receipt mentioning the details of the product. 4. Only warehouse can create the token on the platform on behalf of seller and guarantees the possession of goods. 5. Seller can be anyone assigned by warehouse or warehouse itself. 6. Seller quotes a price for the receipt for which he is interested to sell. 7. Buyer can bid a price he is interested to buy the receipt for, and the maximum bid is recorded on the chain 8. Anyone on the platform can view the bidding details to understand the demand for the receipt. if the seller gets an idea of which bidding is happening, he can change the price according to his understanding. 9. Then if seller like to buy he will buy the receipt for the transaction to happen he	Benefits 1. Easy payment settlements 2. Guaranteed possession of products by warehouse providing trust factor 3. Improved accessibility for small traders 4. Transparency in the system 5. Enhanced supply and trade model for warehouses and traders 6. Easily transferable receipts

need to have enough balance to cover the price of the receipt. 10. The receipt's seller details are changed to new seller who bought the receipt 11. The seller can activate or deactivate the receipt at any time. The seller also can set the price he is willing to sell the receipt. 12. When the owner is willing to take the delivery from warehouse, he enables the token destroy option, through which the token is destroyed by only warehouse who generated that receipt.	
---	--

On-chain and Off-chain data:

We had implemented off-chain data storage in JSON file on the backend server. This is because for every interaction the user is having on the interface, it is not necessary to interact with the smart contract to fetch the data from the chain. We directly synchronized token data using rest API calls. This gives us an edge and solves the problem of scalability when their data is huge on the platform avoid every time unnecessary interaction with the blockchain.

On-chain data we stored all the token details mentioned above the data such as mapping based on the token Id and all the transactional data is stored on the blockchain. We are using events to store the logs of events such as buying the receipt, destroying the receipt, and generating a receipt. These logs store the data about the entire lifetime of an asset that is deployed on the platform. Will be useful for any future investigation or deep dive analysis.

Data stored:

1. struct WarehouseDetails
2. uint256 activeCount
3. mapping(uint256 => WarehouseReceiptDetails) public WarehouseReceipts;
4. mapping(uint256 => uint256) public HighestBid;
5. mapping(uint256 => address) public HighestBidder;

Off-chain data is mainly used to sync the data from every transaction happening on the chain and simultaneously store it on the JSON file. This helps to load and fetch the data for the purpose other than involving the chain. This increases the scalability and decreases the variables which we store on the chain usually not necessarily on the blockchain.

Data stored as JSON format:

```
{
  tokenId
  receiptName,
  price,
  sellerAddress,
  highestBid,
  highestBidder,
  active,
  uri,
  burnit,
  destroy,
}
```

But the advantage we have is that we have robust rules implemented on the smart contract end also to not allow any sort of illegal transactions even if by chance the web interface allowed any sort of that transaction.

This gives us mainly the advantage that we can serve the web interface faster avoiding scalability issues when there are many NFTs launched on this platform and also maintaining the integrity of our system. Most of the data overlap on both ends but some data is not necessarily removed on one of the sides as it does not serve any purpose. The destroy variable is automatically integrated on-chain and does not need to store any variable but on Off-chain we need to store that the receipt never displays on the marketplace.

Design Diagrams

- Use case Diagram



Fig: Use case diagram for warehouse receipts

- **Contract Diagram**

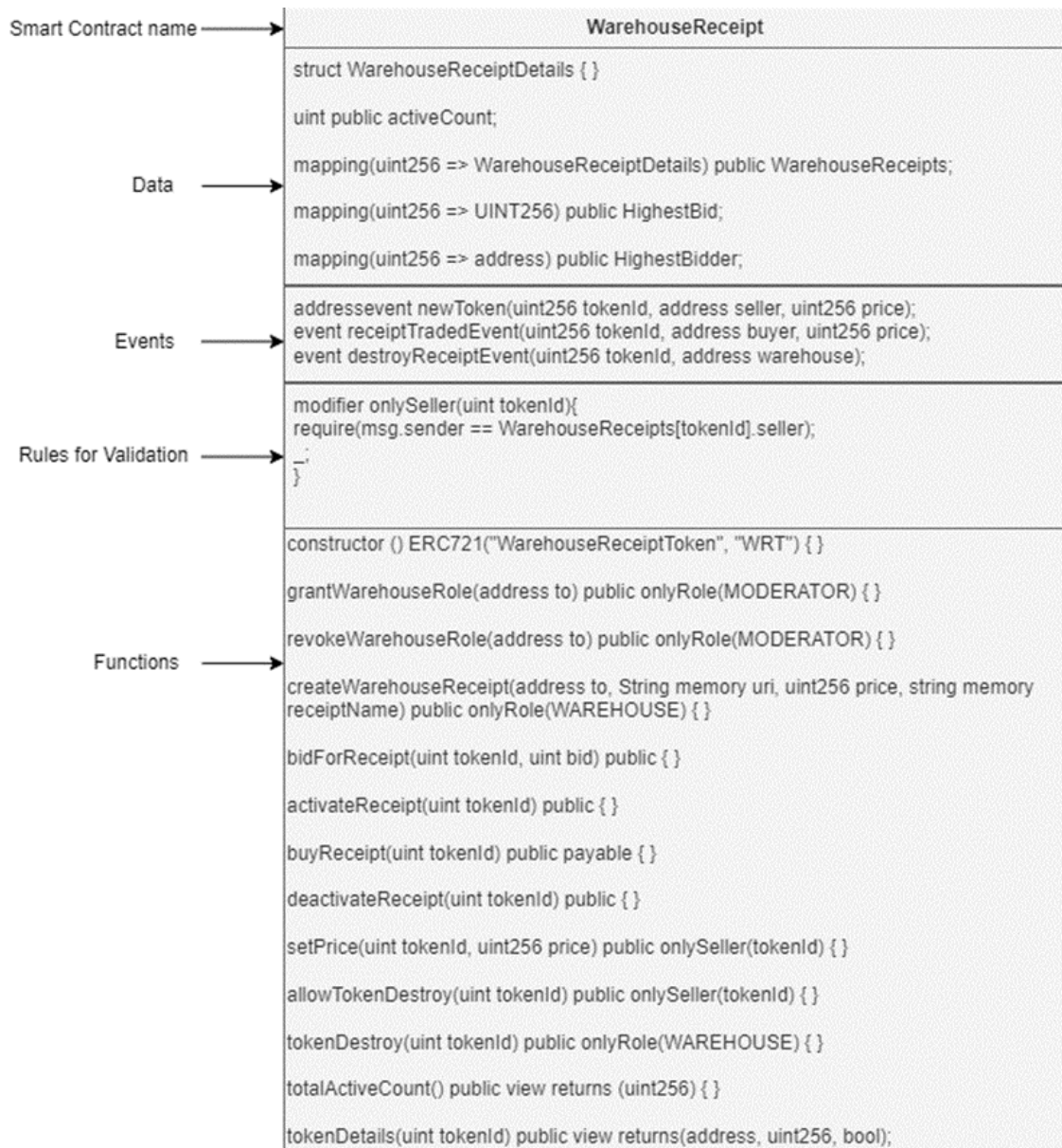
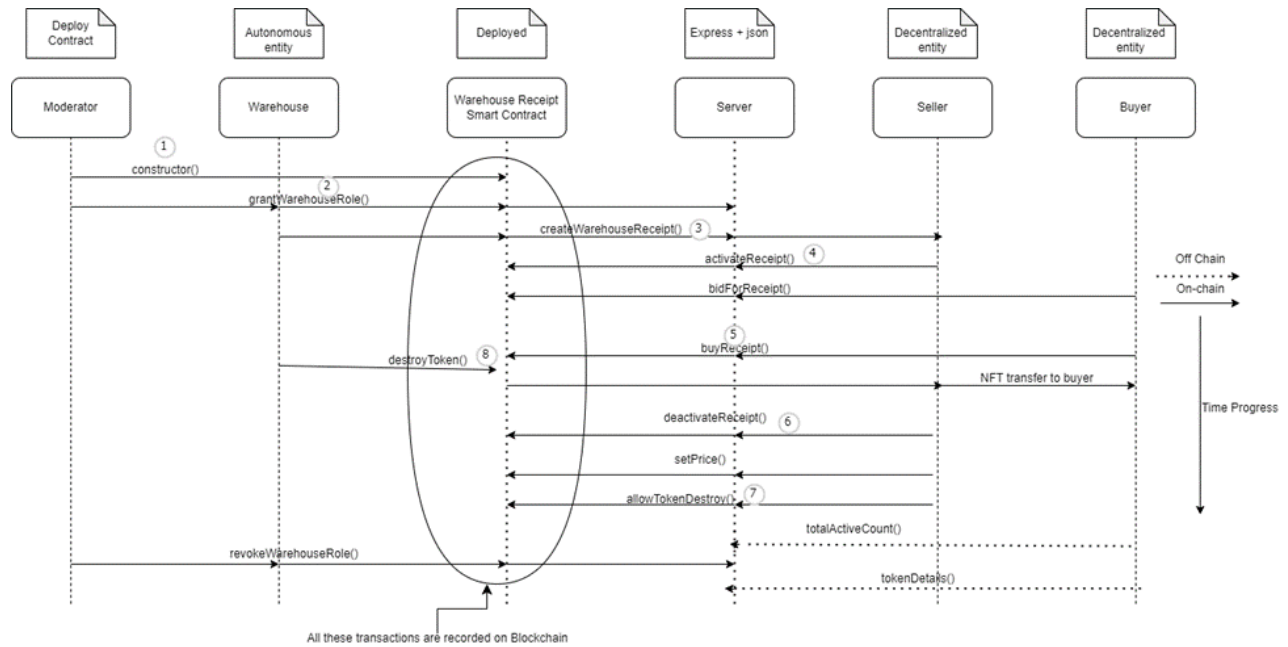
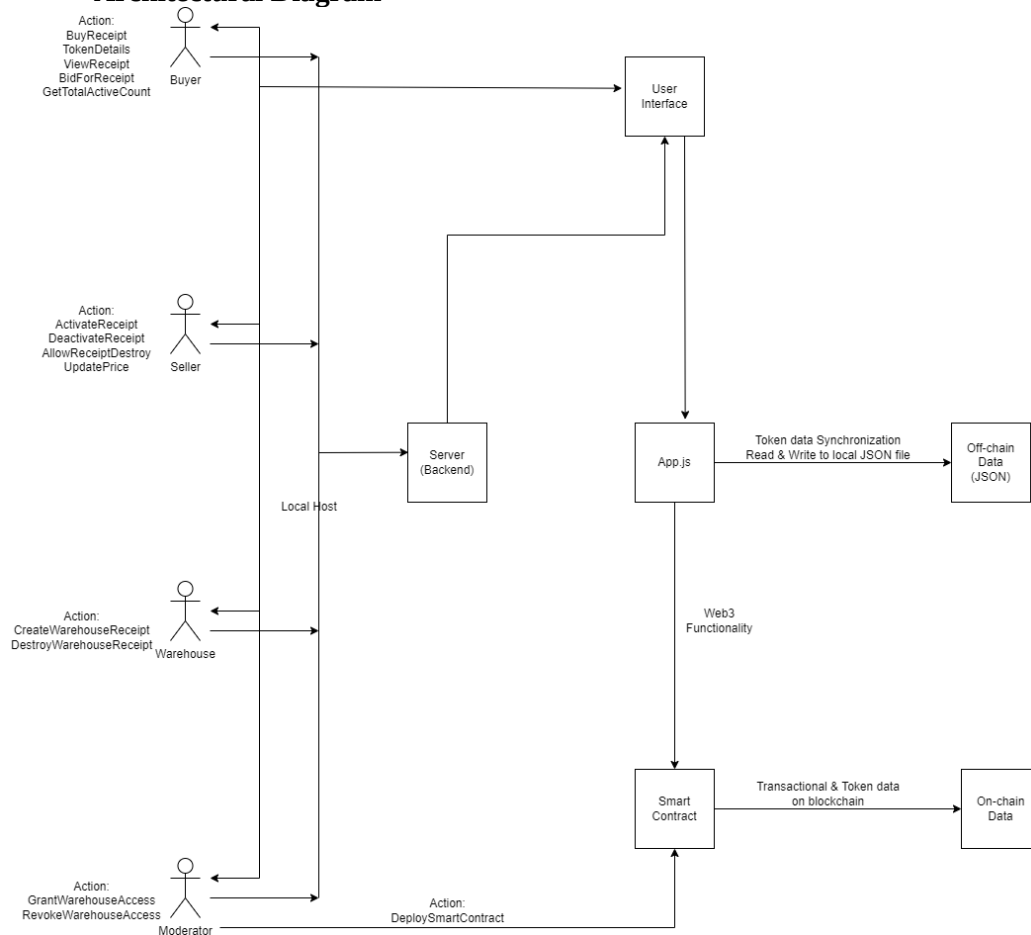


Fig: Contract diagram

• Sequence Diagram



• Architectural Diagram



Instructions(Local):

1. Make sure the ganache is running on your system and import all the accounts provided in the ganache to your MetaMask using mnemonic key.
2. Navigate to warehousereceipt-contract folder.
3. Run the “npm install” command on the terminal inside warehousereceipt-contract folder. In case it doesn't work try install "npm install @openzeppelin/contracts" to install openzeppelin dependencies.
4. Run the “truffle compile” command on the terminal inside warehousereceipt-contract folder.
5. Run the “truffle migrate –reset” command on the terminal inside warehousereceipt-contract folder.
6. This ensures that the smart contract setup is complete.
7. Navigate to warehousereceipt-app folder.
8. Run the “npm install” command on the terminal inside the warehousereceipt-app folder.
9. Run the “npm run start” command on the terminal inside the warehousereceipt-app folder.
10. On browser visit <http://localhost:3000/>. This opens our application on the browser.

Testing Steps:

11. Make sure your MetaMask wallet is connected to the website. In case it is not connected connect it manually.
12. Your first account imported from the ganache to the MetaMask will be the moderator as it is used for deploying the smart contract. So, any sort of moderator actions could be provided by only the first account. This will not limit the account abilities to trade the digital assets on the platform. In case you are using the global link make sure that you have sepolia ETH for doing transaction. You can get Sepolia ETH from alchemy or any other faucet.
13. Now the moderator can provide warehouse access to any user listed on the MetaMask from the ganache by visiting the Moderator tab on the top left corner.
14. A Moderator can also revoke the warehouse access using the same page provided only moderator has the access to grant or revoke the warehouse access. Any other account trying to perform the same actions dedicated to moderator will fail and pops up an alert that the transaction has failed.
15. An account having warehouse access can create a receipt in the name of a seller. The warehouse himself could be a seller in some cases. The details that must be provided during the warehouse receipt creation are as follows: Receipt name, Seller address (EOA), Price of the receipt and Receipt URL.
16. The receipt URL is nothing but the URL to the actual digital warehouse receipt which is generated by the warehouse. This ensures that the receipt is generated by the warehouse and contains all the necessary information related to the product. This is an optional field, and this increases the credibility of the receipt generated on the platform.
17. A warehouse account can destroy the receipt provide the seller of the receipt has to enable the option to destroy ticket by the warehouse. This usually gets enabled during or after the delivery of the goods from the warehouse. Once it is enabled the warehouse can destroy the receipt and it will not be displayed on the platform anymore.
18. A total Active Receipt Count gives the popup how many receipts are active on the platform. Anyone can invoke this function to see total active receipts count.
19. Once you navigate to My Receipts page, we see all the receipts owned by a particular account. A seller can activate or deactivate a particular receipt on this page.
20. A seller can enable the receipt to be destroyed on this page by using token ID, this gives the warehouse the access to destroy the receipt after the delivery.
21. A seller can update the price of the receipt on this page by entering the token ID and the price at which the receipt must be sold.
22. The about section on the top right corner of the page provides the information of the platform.
23. Navigate to Marketplace tab on the top left corner of the page. There you can see all the active receipts listed for trade. By clicking the buy option available on the receipt, you can buy the token

provided you should have enough balance in your account otherwise the transactions will not be successful, and the asset transfer does not take place. If the transaction is successful, you will see a successful notification and the asset gets transferred to you.

24. Using the place bid option on the bottom left corner you should be able to place any the amount you are interested in buying the receipt. If the bid, you placed is higher than the existing highest bid your bid amount will show up on the platform otherwise it will not get updated.
25. By clicking the receipt details option, you will be able to see the details of all the receipts on the platform individually.
26. Sample receipt option on the bottom right corner will redirect to the URL provided by the warehouse while creating the receipt on the platform. If the URL is not provided by default, you will be directed to the homepage of the website which is marketplace.
27. Wait for any transaction to complete and receive the pop up alert the transaction. The confirmation from the on chain transaction usually takes time. It helps in synchronizing the off-chain data successfully.

Smart Contract: Warehouse Receipt

Data:

1. struct WarehouseDetails
This variable store all the details of the warehouse receipt details in a structured format.
2. uint256 activeCount
This variable keeps track of the active count of total number of active tokens on the chain.
3. mapping(uint256 => WarehouseReceiptDetails) public WarehouseReceipts;
This variable maps the tokenId to the warehouse receipt details variable using mapping data structure which makes it easily accessible based on the receipt number.
4. mapping(uint256 => uint256) public HighestBid;
This mapping variable keeps track of the individual receipt's highest bidder based on the receipt number.
5. mapping(uint256 => address) public HighestBidder;
This mapping variable keeps track of the highest bidder of the individual receipt based on the receipt number.

Events:

1. newToken(uint256 tokenID, address to, uint256 price)
Emits event when a new token is minted on the chain used for tracking token ids on the UI and used to store the logs of minted tokens.
2. receiptTradedEvent(uint256 tokenID, address buyer, uint256 price)
This event emits when the receipt is bought by any account. This keeps track of all the traded transactions on the chain for any future reference or investigation.
3. destroyReceiptEvent(uint256 tokenID, address warehouse)
This event emits when the receipt is destroyed, and by which warehouse account the receipt has been destroyed.

Modifiers:

1. `onlySeller(uint256 tokenID)`
For particular functions need to be only limited to seller of the token in those case we use this modifier so that only seller of the receipt can perform those activities.

Functions:

1. `constructor()`
This will run once the smart contract is deployed and assign the user who deployed it as moderator. Also initiates the receipt value to 1.
2. `grantWarehouseRole(address to)`
This function is only accessible to moderator and used to grant permission to any user as warehouse so that they can create new receipts on the platform.
3. `revokeWarehouseRole(address to)`
This function is only accessed by moderator as it is used to revoke warehouse privilege of a warehouse user.
4. `createWarehouseReceipt(address to, string memory uri, uint256 price, string memory receiptName)`
This function is used only by the warehouse user to generate a new receipt containing all the details provided for receipt like receipt name, seller address, price of the receipt and URL to the original warehouse receipt.
5. `bidForReceipt(uint tokenId, uint bid)`
This function can be invoked by any user except the seller of the receipt to put a bid on the respective receipt. If the bid provided is higher than the existing bid, it will get updated.
6. `activateReceipt(uint tokenId)`
When an account buys a receipt, it turns inactive. It will be ready for trade only when the receipt is turned active by the owner of the receipt and visible on the marketplace.
7. `buyReceipt(uint tokenId)`
This receipt is called by any user except the seller to buy the receipt. The only condition is that buyers need to possess enough balance to buy the receipt otherwise the transaction will be unsuccessful, and the asset transfer does not happen.
8. `deactivateReceipt(uint tokenId)`
If a receipt owner is not interested in selling a receipt, he can change the receipt status to inactive so that it will not be visible on the marketplace.
9. `setPrice(uint tokenId, uint price)`
The owner of a receipt can changes the price he is expecting to sell using this function depending on the bid amount he is receiving. He can either decrease or increase the price based on the demand for the receipt.
10. `allowTokenDestroy(uint tokenId)`
This is only executed by the receipt owner when he is about to take the delivery from the warehouse so that the warehouse owner gets the access to destroy the token after the product delivery.

11. `destroyToken(uint tokenId)`
This is only executed by the warehouse user who generated the receipt only after it was enabled to destroy by the seller of the receipt.
12. `totalActiveCount()`
This function provides the total count of the active receipts traded on the platform.
13. `tokenDetails(uint tokenId)`
This function provides the details of the receipts on the platform.

Deployment Instructions:

1. First create an account in Infura and there we get an API key which we need to use in our project for smart contract deployment. Sample Infura link looks like “https://sepolia.infura.io/v3/<api-key>”.
2. Add .env file in the warehousereceipt-contract folder with the following information:
 - a. `INFURA_API_KEY = “https://sepolia.infura.io/v3/<api-key>”`.
 - b. `MNEMONIC = “metamask secret phase of moderator”`
3. Navigate to truffle-config.js file and add the following content:

```
require('dotenv').config();
const { MNEMONIC, INFURA_API_KEY } = process.env;
const HDWalletProvider = require('@truffle/hdwallet-provider');
```

within network add:

```
sepolia: {
  provider: () => new HDWalletProvider(MNEMONIC, INFURA_API_KEY),
  network_id: '11155111',
  gas: 4465030
}
```
4. Open terminal on the local system.
5. Navigate to the warehousereceipt-contract folder.
6. Run the “truffle compile”.
7. Run the “truffle migrate --network sepolia”.
8. As an output we get smart contract address on which the infura blockchain is deployed. Make note of the address to integrate it to the web app. The smart contract is successfully deployed on the infura.
9. Navigate to warehousereceipt-app root directory, run “git init” to initialize a git repository.
10. Run git add to stage all the files that need to be committed.
11. Run git commit -m “message” to commit your changes to the remote git repository.
12. Run git push to push your committed changes to remote repository.
13. Navigate to warehousereceipt-app folder. Run “Heroku login”.
14. Run for first time “Heroku create <appname>”.
15. Create Procfile in the root folder of warehousereceipt-app.
16. Add the following command to the Procfile: “web: node index.js”.

17. Commit and push the Procfile to the git repository.
18. Run in the terminal inside the root folder of warehousereceipt-app. Run the “git push Heroku main”.
19. Open the Heroku dashboard where you will find the open the app and it will redirect you to the global URL of your DAPP.
20. We also find the build status for every commit change we push to the git repository.
21. In app.js we need to replace the contract address variable with the smart contract address we got during the deployment on the infura.
22. We also replace the localhost URL in app.js and index.js with global link provided by the Heroku.
23. Now commit the following changes and push them to the git repo. The Heroku will automatically deploy your changes to fresh build based on your changes.
24. Access the DAPP using the link provided by Heroku globally.

Digital Warehouse Receipt NFT Link: <https://warehousereceipt-nft.herokuapp.com/>

References:

1. <https://www.manning.com/books/blockchain-in-action>
2. <https://docs.openzeppelin.com/contracts/4.x/>
3. <https://www.foley.com/en/insights/publications/2022/02/blockchain-for-digital-logistics-smart-warehouses>
4. <https://docs.soliditylang.org/en/v0.8.19/>
5. <https://www.investopedia.com/terms/w/warehousereceipt.asp>
6. <https://www.wallstreetmojo.com/warehouse-receipt/#warehouse-receipt-system>
7. <https://www.alamy.com/stock-photo/warehouse-receipt.html?sortBy=relevant>
8. <https://eips.ethereum.org/EIPS/eip-721>
9. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
10. <https://nodejs.org/dist/latest-v18.x/docs/api/>

Approved By: Ram Kashyap